

Del bibliotecario solitario al equipo de agentes

Una línea de tiempo de las arquitecturas agénticas, de RAG a los sistemas multiagente
(2020–2026)

Matías Adell

Seminario de Tendencias de Ciencia de Datos — 2026

Resumen

Entre 2020 y 2026, la forma de conectar modelos de lenguaje con información externa pasó de un pipeline fijo de dos pasos — recuperar y generar — a redes de agentes que deciden, razonan, se corrigen entre sí y negocian con agentes de otras organizaciones. Este artículo recorre esa evolución como una línea de tiempo de 24 arquitecturas, organizadas en seis eras narrativas: el RAG clásico y sus límites, la aparición de agentes que razonan y actúan, el momento en que el propio RAG se vuelve agéntico, la coordinación entre múltiples agentes, los protocolos que permiten que agentes de distintos proveedores se entiendan, y los primeros intentos de sistematizar todo el campo en una taxonomía formal. Cada arquitectura se ancla en el paper que la originó — verificado, no simplemente el más citado — y el artículo cierra con una lectura, explícitamente marcada como prospectiva, de hacia dónde parece dirigirse el campo: gobernanza entre agentes, seguridad multiagente, infraestructura para una “web de agentes”, y arquitecturas que se diseñan a sí mismas.

Índice

1. Introducción	2
2. Era 1 — RAG clásico y sus límites (2020–2022)	3
2.1. RAG: retrieval-augmented generation (2020)	3
2.2. HyDE: el puente hacia la mejora de queries (2022)	3
3. Era 2 — Agentes: razonar y actuar (2022–2023)	5
3.1. ReAct: razonar + actuar (2022)	5
3.2. Reflexion: aprender del propio error sin reentrenar (2023)	6
3.3. Generative Agents: memoria y planificación (2023)	6
4. Era 3 — El RAG se vuelve agéntico (2023–2024)	7
4.1. Self-RAG: recuperar, generar y autocriticarse (2023)	7
4.2. MemGPT: RAG con memoria persistente (2023)	8
4.3. Modular RAG: de pipeline a piezas de LEGO (2023–2024)	8
4.4. Corrective RAG (CRAG): no confiar ciegamente en lo recuperado (2024)	9
4.5. RAPTOR: retrieval jerárquico en árbol (2024)	9
4.6. Adaptive-RAG: no toda query necesita la misma estrategia (2024)	10
4.7. GraphRAG: recuperar sobre relaciones, no solo similitud (2024)	10
4.8. Cache-Augmented Generation (CAG): a veces ni hace falta recuperar (2024)	11
4.9. Vectorless RAG / PageIndex: recuperar sin embeddings (2025)	11

5. Era 4 — Arquitecturas multiagente (2023–2025)	12
5.1. CAMEL: el primer diálogo nativo entre dos agentes LLM (2023)	12
5.2. AutoGen: conversación multiagente como framework (2023)	12
5.3. MetaGPT: procedimientos operativos estándar para agentes (2023)	13
5.4. AgentVerse: dinámica de grupo emergente (2023)	14
5.5. TalkHier: supervisores que supervisan supervisores (2025)	14
5.6. Mixture-of-Agents: ensamble multiagente por capas (2024)	15
6. Era 5 — Protocolos de interoperabilidad (2024–2026)	15
6.1. Model Context Protocol (MCP): estandarizar agente↔herramientas (2024)	15
6.2. Agent2Agent (A2A): estandarizar agente↔agente (2025–2026)	16
7. Era 6 — El campo se sistematiza (2025–2026)	17
7.1. Agentic RAG: primera taxonomía formal (2025)	17
7.2. SoK: Agentic RAG como proceso de decisión (2026)	17
8. Hacia dónde va esto	18
9. Conclusión	18

1. Introducción

RAG (*Retrieval-Augmented Generation*) resuelve el problema del corte de conocimiento de un LLM combinando una etapa de retrieval — búsqueda semántica sobre un índice, típicamente vectorial — con generación condicionada por el contexto recuperado. En 2020 ese pipeline era rígido: dos fases, un único pase, sin mecanismo de control. Esta línea de tiempo recorre cómo cada arquitectura posterior fue removiendo esa rigidez, en orden: primero dotando al modelo de agencia sobre su propio proceso de razonamiento y acción (ReAct, 2022), después aplicando esa agencia directamente al pipeline de RAG — decidir cuándo recuperar, evaluar si lo recuperado alcanza, reformular la query (2023–2024) —, después coordinando múltiples agentes especializados bajo topologías de orquestación distintas (2023–2025), y finalmente estandarizando la interoperabilidad entre agentes de proveedores distintos vía protocolos abiertos (2024–2026).

Es un campo emergente en sentido literal: la mayoría de los trabajos citados tienen menos de tres años, y el más reciente es de 2026. Por eso el artículo se estructura como línea de tiempo y no como catálogo de técnicas: cada arquitectura resuelve una limitación puntual de la anterior, y esa cadena causal se pierde si se presentan fuera de orden. La Figura 1 resume las seis eras narrativas; una séptima sección, marcada explícitamente aparte, discute señales prospectivas sobre hacia dónde converge el campo — sin el mismo estándar de evidencia que el resto del artículo.

Cada arquitectura se ancla en el paper que la originó, verificado individualmente contra el consenso de la literatura (no el más citado, ni el más reciente que la menciona). Cuando no existe un paper académico único — dos de los protocolos de la Era 5 — se cita la especificación técnica correspondiente y se lo señala explícitamente en el texto. El detalle de implementación de un pipeline de RAG estándar (chunking, modelos de embedding, bases vectoriales, hybrid search) se asume conocido y no se repite acá; ver el material de referencia¹ para ese nivel de detalle.

¹01_material-base/Summary.pdf de este proyecto.

Cómo leer este artículo

Cada arquitectura se presenta con la misma estructura: qué limitación concreta de la arquitectura anterior resuelve, cuál es su mecanismo central, y qué habilita en la arquitectura siguiente — de modo que el timeline se lea como una cadena causal, no como fichas técnicas independientes.

2. Era 1 — RAG clásico y sus límites (2020–2022)

Esta era cubre la formalización original de RAG y su primera limitación estructural expuesta: el desacople sintáctico entre query y documento relevante.

2.1. RAG: retrieval-augmented generation (2020)

Un LLM tiene su conocimiento fijado en el momento del entrenamiento, sin acceso nativo a información posterior ni a datos privados o específicos de un dominio. Lewis et al. [2020] formalizaron RAG como la combinación de un retriever no paramétrico — un índice denso sobre embeddings de los documentos, consultado por similitud — con un modelo generativo condicionado por los pasajes recuperados, en vez de depender únicamente del conocimiento paramétrico almacenado en los pesos del modelo.

La propiedad relevante de este pipeline (Figura 2) para el resto del artículo no es su implementación interna — cubierta en el material de referencia — sino su rigidez: no hay decisión sobre si conviene recuperar, ninguna evaluación de si el resultado recuperado es relevante, ni posibilidad de reformular la query si la primera búsqueda falla. La Era 3 es, en sustancia, la eliminación sucesiva de esas tres rigideces.

Fuente: [Lewis et al., 2020]

2.2. HyDE: el puente hacia la mejora de queries (2022)

La búsqueda por similitud vectorial tiene un problema de *asimetría query-documento*: una pregunta y el pasaje que la responde no son necesariamente cercanos en el espacio de embeddings, aun siendo semánticamente relevantes entre sí, porque difieren en estructura sintáctica — una pregunta no está fraseada como una respuesta. Gao et al. [2023a] invierten el pivote de búsqueda: en vez de embeber la query, instruyen a un LLM a generar una respuesta hipotética a la pregunta — sin verificarla, puede contener errores factuales — y embeben esa respuesta hipotética para hacer la búsqueda contra el índice. Una respuesta hipotética, aunque no verificada, es estilísticamente más cercana a un pasaje real que la query original, lo que mejora la precisión del retrieval sin fine-tuning del encoder ni del retriever.

El aporte conceptual de HyDE (Figura 3) para el resto del artículo no es la técnica en sí, sino el patrón que instala: el LLM interviene activamente en el propio pipeline de retrieval, en vez de ser un consumidor pasivo de lo que el retriever le entrega. Ese patrón — agencia sobre el propio proceso — es el precursor directo de ReAct (Era 2) y, más adelante, del control explícito sobre retrieval que exploran Self-RAG y Corrective RAG (Era 3).

Fuente: [Gao et al., 2023a]

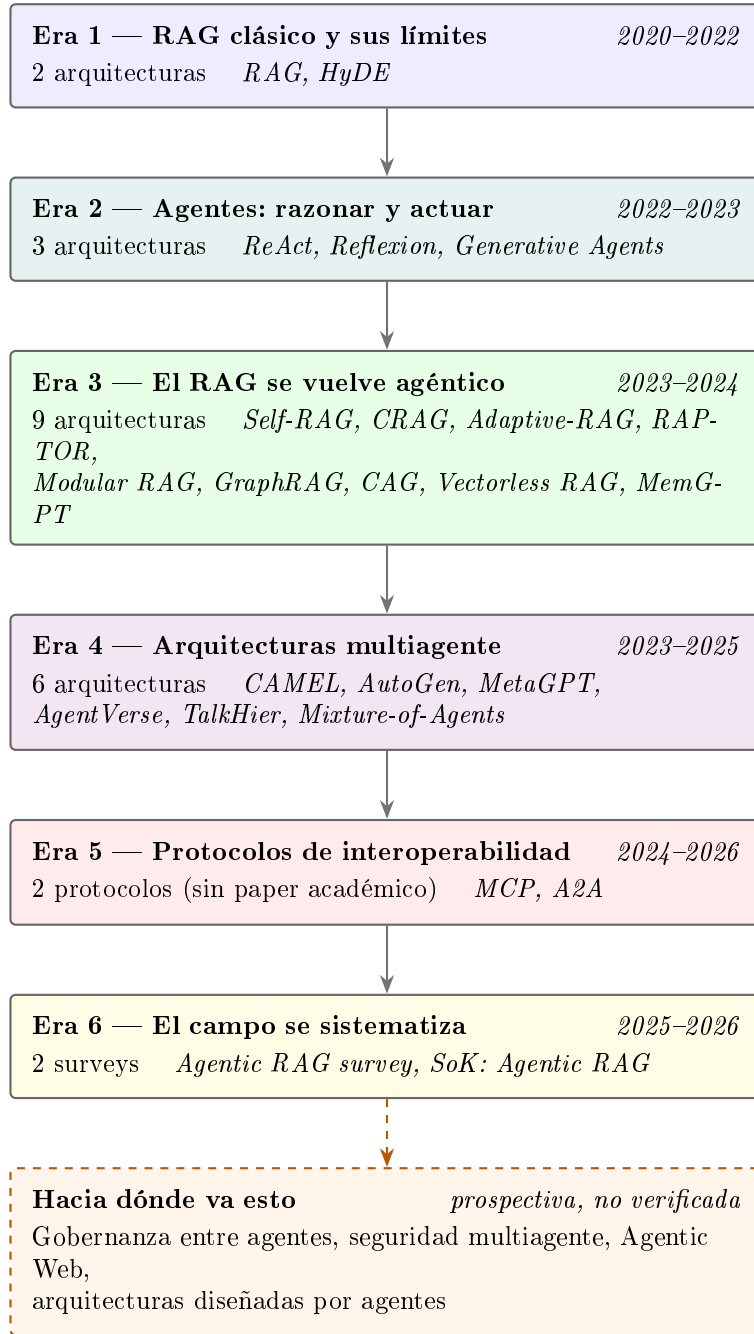


Figura 1: Línea de tiempo de arquitecturas agénticas y multiagente, 2020–2026. Cada caja agrupa una era narrativa (Secciones 2–7); la caja punteada final corresponde a la Sección 8, explícitamente prospectiva.

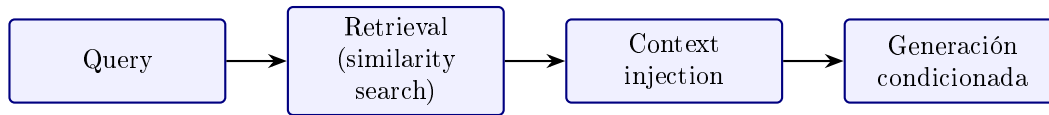


Figura 2: Pipeline original de RAG: dos fases con escalas de tiempo distintas — indexing offline y query time online — ejecutadas siempre en el mismo orden, sin mecanismo de control sobre si conviene recuperar o si el resultado recuperado es adecuado.

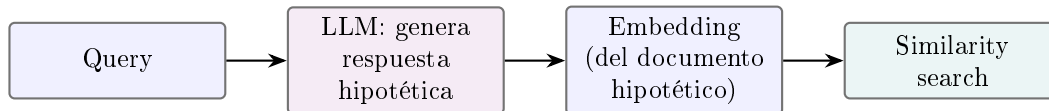


Figura 3: HyDE: la query nunca se embebe directamente — se embebe la respuesta hipotética que el LLM genera a partir de ella, y esa embedding es la que se usa para buscar en el índice.

3. Era 2 — Agentes: razonar y actuar (2022–2023)

HyDE introdujo agencia sobre un único paso del pipeline (la formulación de la query). Esta era generaliza esa idea a nivel de *control loop* completo, todavía sin acoplarla a RAG: un **agente** es un sistema LLM-céntrico que decide su propia secuencia de acciones — vía tool calling — en vez de ejecutar un pipeline hardcodeado.

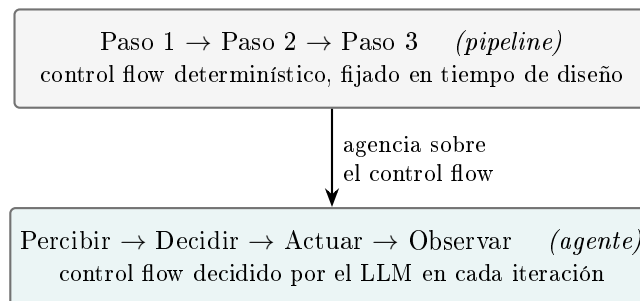


Figura 4: Un pipeline ejecuta un control flow fijo; un agente lo decide dinámicamente, iteración por iteración.

3.1. ReAct: razonar + actuar (2022)

Reasoning (chain-of-thought) y acting (tool use) se trataban como capacidades separadas: un modelo podía generar una traza de razonamiento antes de responder, o podía invocar herramientas, pero no había interleaving entre ambos — sin traza de razonamiento, una acción fallida no se podía diagnosticar; sin grounding en observaciones reales, el razonamiento podía divergir sin corrección. Yao et al. [2023] proponen intercalar ambos en un mismo loop: el modelo genera un *thought*, ejecuta una *action* (tool call) condicionada por ese thought, recibe una *observation*, y repite hasta emitir una respuesta final.

Este loop (Figura 5) es el patrón fundacional de prácticamente cualquier agente LLM posterior; toda arquitectura multiagente de la Era 4 es, en el fondo, una composición de instancias de este mismo loop coordinadas entre sí.

Fuente: [Yao et al., 2023]

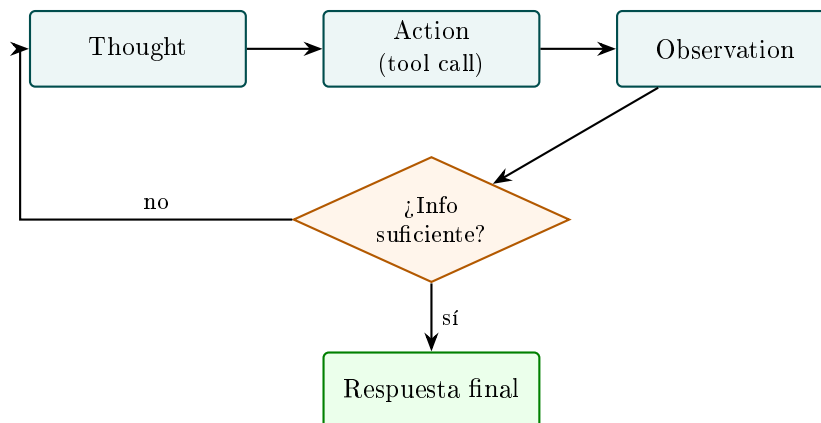


Figura 5: Loop de ReAct: thought–action–observation, iterado hasta que el propio modelo determina que la evidencia acumulada alcanza.

3.2. Reflexion: aprender del propio error sin reentrenar (2023)

ReAct provee acting y observing, pero ningún mecanismo de aprendizaje inter-episodio: un fallo en el trial t no informa al trial $t + 1$ si nada en el estado del agente cambió. Shinn et al. [2023] proponen *refuerzo verbal*: en vez de actualizar pesos vía gradiente, el agente genera una crítica en lenguaje natural de su propio fallo — qué salió mal, qué estrategia probar distinto — y la persiste en un buffer de memoria episódica que se inyecta como contexto adicional en el siguiente intento.

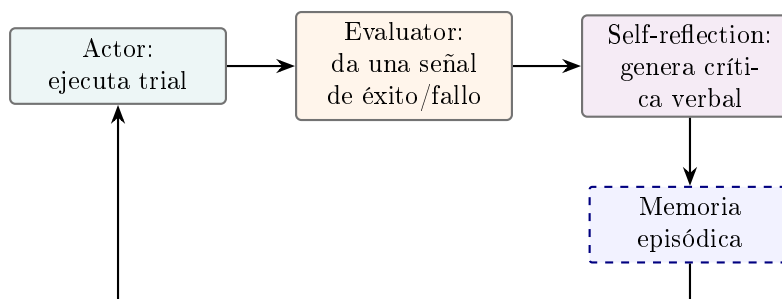


Figura 6: Loop de Reflexion: el Evaluator juzga el trial, el modelo de self-reflection lo traduce a una crítica verbal, y esa crítica se persiste en memoria episódica para condicionar el próximo trial del Actor — sin actualizar ningún parámetro.

El mecanismo de autocritica de Reflexion (Figura 6) — generar una señal de evaluación sobre la propia salida y usarla para condicionar el siguiente paso — es el componente que retoman Self-RAG y Corrective RAG (Era 3) aplicado específicamente al output del pipeline de retrieval.

Fuente: [Shinn et al., 2023]

3.3. Generative Agents: memoria y planificación (2023)

Los agentes anteriores resuelven tareas puntuales sin estado persistente entre interacciones. Park et al. [2023] introducen una arquitectura de memoria de tres capas sobre una población de 25 agentes en un entorno simulado: un *memory stream* que registra cada observación en lenguaje natural con timestamp, un mecanismo de recuperación que puntúa memorias por recencia, relevancia (similitud de embeddings con la query actual) e importancia (puntuada por el propio LLM), y una capa de

reflection que sintetiza periódicamente memorias de bajo nivel en observaciones de más alto nivel, usadas a su vez para planificar. De esa combinación emergieron comportamientos multi-agente no programados explícitamente (coordinación espontánea de una celebración grupal, entre otros).

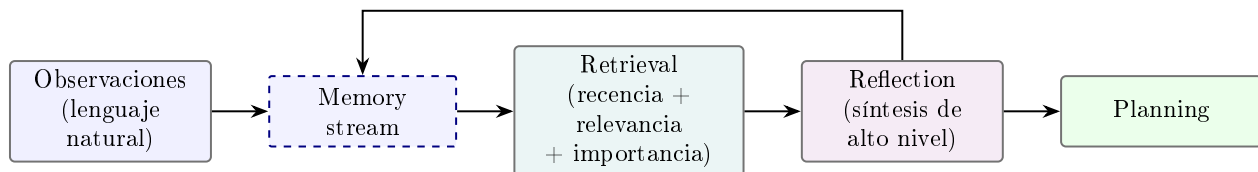


Figura 7: Arquitectura de memoria de Generative Agents: las observaciones crudas se acumulan en el memory stream, se recuperan según un score compuesto, se sintetizan en reflexiones de más alto nivel — que vuelven a escribirse al stream — y alimentan la planificación.

Esta arquitectura de memoria de tres capas es la que adopta MemGPT (Era 3) para resolver persistencia en RAG, y el mecanismo de base para cualquier agente de la Era 4 que necesite mantener estado sobre interacciones pasadas con otros agentes.

Fuente: [Park et al., 2023]

4. Era 3 — El RAG se vuelve agéntico (2023–2024)

RAG (Era 1) y el control loop agéntico (Era 2) convergen: cada arquitectura de esta sección le devuelve al agente el control sobre una de las tres rigideces identificadas en la Era 1 — si conviene recuperar, si lo recuperado es adecuado, con qué profundidad iterar. Con nueve arquitecturas, es la era más numerosa del artículo.

4.1. Self-RAG: recuperar, generar y autocriticarse (2023)

RAG estándar recupera incondicionalmente en cada query y nunca evalúa la salida generada contra la evidencia recuperada. Asai et al. [2024] entrenan un único LM para generar, intercalados con el texto, *reflection tokens* discretos: **Retrieve** decide si conviene recuperar en ese punto de la generación; **ISREL**, **ISSUP** e **ISUSE** evalúan — post-hoc, sobre cada segmento generado — si el segmento está soportado por el pasaje recuperado, si es relevante y si es útil. La inferencia usa estos tokens para hacer beam search segmento a segmento, seleccionando la continuación mejor calibrada.

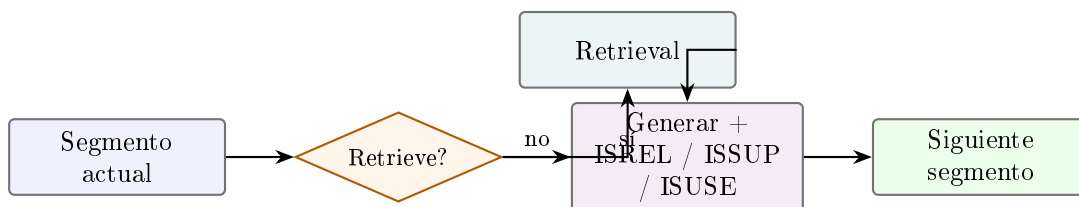


Figura 8: Self-RAG decide, segmento a segmento, si conviene recuperar (**Retrieve**), y anota cada continuación generada con tokens de reflexión (**ISREL/ISSUP/ISUSE**) usados después para el beam search.

Self-RAG (Figura 8) es la bisagra de la era: es el primer trabajo que resuelve, en un mismo modelo, tanto *cuándo* recuperar como la *evaluación post-hoc* de la generación contra la evidencia — las dos capacidades que el resto de esta sección desarrolla por separado y con más profundidad.

Fuente: [Asai et al., 2024]

4.2. MemGPT: RAG con memoria persistente (2023)

RAG estándar es stateless entre queries: no hay mecanismo nativo para persistir contexto conversacional más allá de la ventana de contexto actual. Packer et al. [2023] adaptan el paginado de memoria virtual de los sistemas operativos: un *main context* (análogo a RAM) siempre presente en el prompt, y un *external context* (análogo a disco, archival + recall storage) del que se recupera información bajo demanda. El propio LLM emite function calls para mover datos entre ambos niveles — decide qué archivar, cuándo recuperar, cuándo generar una interrupción para resumir contexto que se está por desbordar.

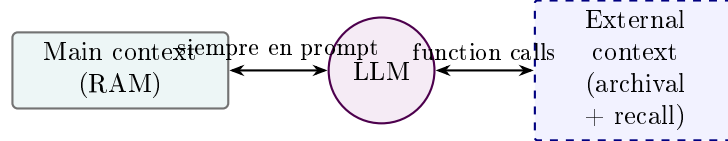


Figura 9: MemGPT: el LLM decide, vía function calls propias, qué mover entre el contexto de trabajo (siempre en el prompt) y el almacenamiento externo (recuperado bajo demanda) — paginado de memoria virtual aplicado al propio agente.

Con MemGPT (Figura 9), la persistencia conversacional se resuelve con el mismo mecanismo de retrieval que RAG aplica a documentos externos, aplicado acá a un store de memoria propio del agente.

Fuente: [Packer et al., 2023]

4.3. Modular RAG: de pipeline a piezas de LEGO (2023–2024)

Gao et al. [2023b] formalizan la taxonomía Naive RAG / Advanced RAG / Modular RAG: en vez de un pipeline lineal fijo, RAG se descompone en módulos de retrieval, generación y augmentation intercambiables, orquestados con mecanismos explícitos de routing, scheduling y fusion — permitiendo topologías lineales, condicionales, en branching o con loops, según el caso de uso.

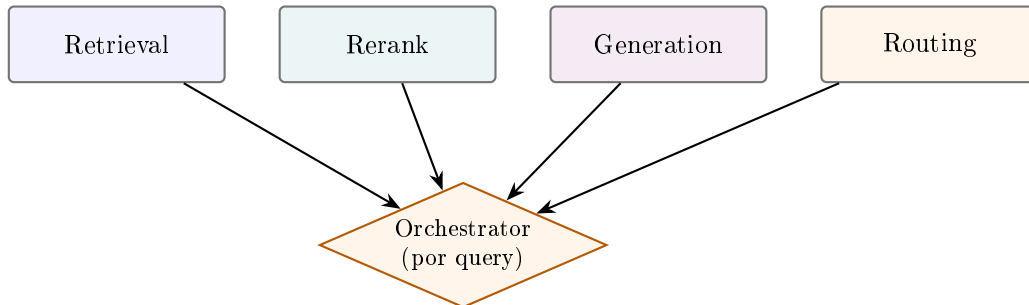


Figura 10: Modular RAG: retrieval, rerank, generation y routing son módulos intercambiables; el orchestrator los combina — lineal, condicional, en paralelo o en loop — según lo que requiera cada query.

Esta taxonomía (Figura 10) no es una arquitectura adicional en la lista: es el marco bajo el cual el resto de las arquitecturas de esta sección son, literalmente, módulos distintos insertables en el mismo pipeline componible.

Fuente: [Gao et al., 2023b], ampliado en [Gao et al., 2024]

4.4. Corrective RAG (CRAG): no confiar ciegamente en lo recuperado (2024)

Si el retriever recupera pasajes irrelevantes, RAG estándar genera igual, condicionado por evidencia inválida. Yan et al. [2024] insertan un *retrieval evaluator* liviano (un T5 fine-tuneado) que puntúa la relevancia de cada pasaje recuperado y dispara una de tres acciones.

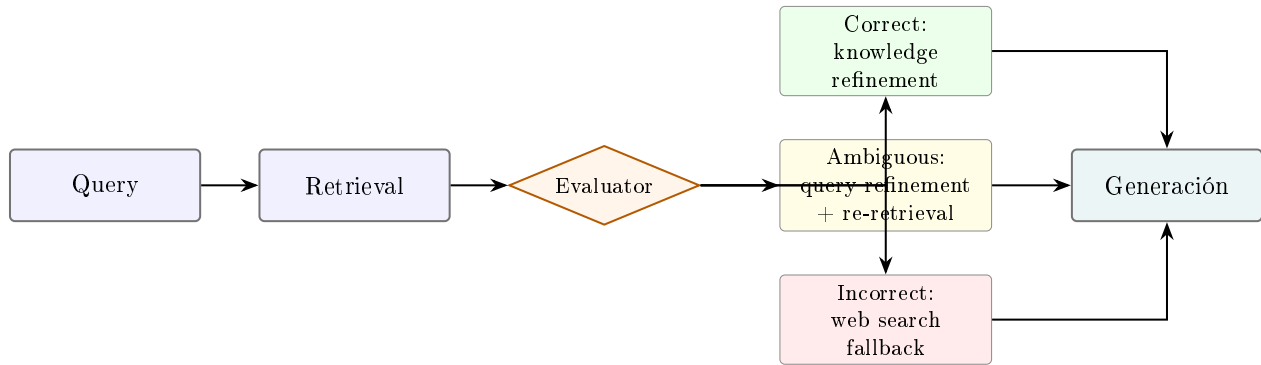


Figura 11: CRAG inserta un evaluador de relevancia entre retrieval y generación; el veredicto (correct/ambiguous/incorrect) determina si el contexto se usa tal cual, se refina localmente, o se descarta en favor de búsqueda web.

Sobre el caso *correct*, CRAG además aplica un algoritmo de decompose-then-recompose: parte cada pasaje en *knowledge strips*, filtra las irrelevantes, y recompone solo las relevantes — reduciendo ruido incluso en el caso de mayor confianza.

Fuente: [Yan et al., 2024]

4.5. RAPTOR: retrieval jerárquico en árbol (2024)

El chunking plano recupera fragmentos cortos que no capturan estructura de documentos largos — preguntas que requieren síntesis global no se responden con un único chunk. Sarthi et al. [2024] construyen, además del índice de chunks, un árbol de resúmenes: clustering (mediante GMM soft clustering) de chunks semánticamente relacionados, resumen de cada cluster, y repetición recursiva del proceso sobre los resúmenes hasta la raíz. Retrieval consulta el árbol completo (*collapsed tree*), agregando nodos de cualquier nivel de abstracción según lo requiera la query.

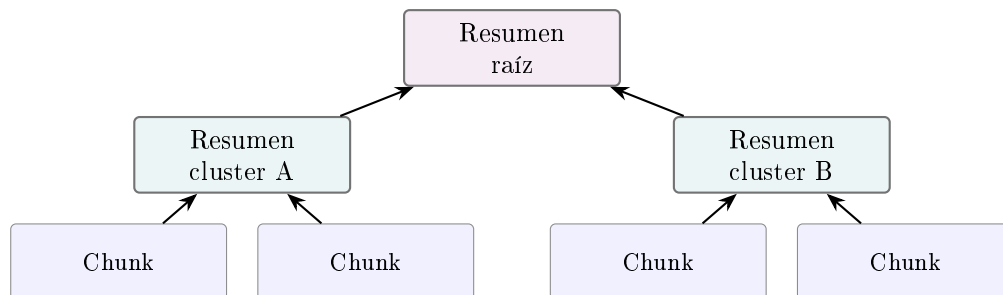


Figura 12: RAPTOR construye un árbol de resúmenes vía clustering + resumen recursivo; retrieval agrega nodos de cualquier nivel — chunk crudo o resumen de distinto grado de abstracción — según lo requiera la query.

Nota de terminología (Figura 12): el material del curso usa “jerárquica” tanto para esto —

jerarquía de *resúmenes de contenido* — como, en la Era 4, para jerarquía de *control entre agentes*. Son estructuras análogas (ambas organizan información en niveles), pero arquitecturas distintas.

Fuente: [Sarathi et al., 2024]

4.6. Adaptive-RAG: no toda query necesita la misma estrategia (2024)

Ninguna arquitectura anterior condiciona su costo computacional a la complejidad real de la query. Jeong et al. [2024] entrenan un clasificador liviano (un LM chico) para estimar la complejidad de la query — con labels recolectados automáticamente a partir de los outcomes observados de distintas estrategias sobre datasets de QA — y enrutar en consecuencia.

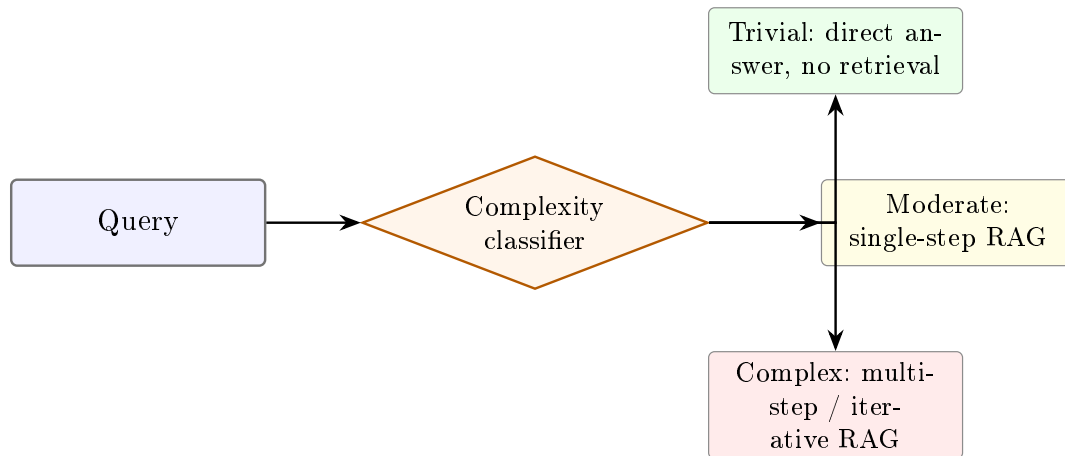


Figura 13: Adaptive-RAG enruta la query, antes de recuperar, a la estrategia de retrieval proporcional a su complejidad.

Adaptive-RAG y CRAG son complementarias, no competidoras: Adaptive-RAG decide *ex ante* qué estrategia usar; CRAG evalúa *ex post* si el resultado de esa estrategia es confiable. Un sistema de producción típico combina ambas: routing primero, corrección después.

Fuente: [Jeong et al., 2024]

4.7. GraphRAG: recuperar sobre relaciones, no solo similitud (2024)

La búsqueda por similitud vectorial no captura relaciones explícitas entre entidades — una query sobre conexiones (“¿qué empresa adquirió a esta otra, y quién la fundó?”) no se responde bien con similitud semántica sobre texto plano. Edge et al. [2024] construyen, vía extracción con LLM, un knowledge graph de entidades y relaciones a partir del corpus, detectan comunidades (algoritmo de Leiden) sobre ese grafo, y pregeneran resúmenes jerárquicos por comunidad. Queries globales se resuelven vía map-reduce sobre los resúmenes de comunidad; queries locales, vía traversal directo del grafo.

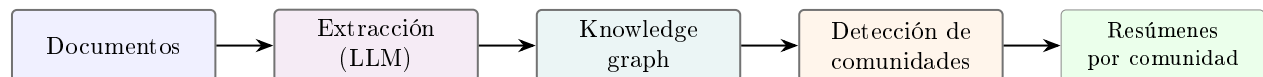


Figura 14: Pipeline de indexado de GraphRAG. Query time: queries locales recorren el grafo directamente; queries globales resuelven vía map-reduce sobre los resúmenes de comunidad pregenerados.

Salvedad (Figura 14): RAG sobre knowledge graphs no se origina en este paper — hay antecedentes de 2023 sin este nombre — pero es el trabajo que acuña “GraphRAG” como término propio, adoptado después por el resto de la industria.

Fuente: [Edge et al., 2024]

4.8. Cache-Augmented Generation (CAG): a veces ni hace falta recuperar (2024)

Toda arquitectura anterior de esta era asume que hay que recuperar en algún momento; la pregunta era solo cuándo y con qué profundidad. Chan et al. [2025] eliminan el paso de retrieval directamente para corpus acotados: si el corpus completo entra en la ventana de contexto, se precomputa una única vez la KV cache de atención sobre ese contexto completo, y se reusa esa cache — sin recomputarla — en cada query subsiguiente.

RAG tradicional: index + similarity search en **cada** query

CAG: precomputar KV cache **una sola vez**, reusar en cada query

Figura 15: CAG reemplaza el retrieval por query por una precomputación única de la KV cache de atención sobre el corpus completo — válido solo mientras el corpus entra en contexto y no cambia con frecuencia.

Es un trade-off (Figura 15), no una mejora estrictamente dominante: CAG elimina la latencia de retrieval por query, pero no escala más allá de lo que entra en contexto ni tolera corpus que cambian con frecuencia suficiente como para justificar recomputar la cache. Es el punto exactamente opuesto, en el espacio de diseño, al problema que motivó RAG originalmente.

Fuente: [Chan et al., 2025]

4.9. Vectorless RAG / PageIndex: recuperar sin embeddings (2025)

Toda esta era asumió, hasta acá, que retrieval implica comparar vectores de embeddings. VectifyAI cuestiona esa premisa: un documento con estructura explícita (secciones, tabla de contenidos) admite navegación por razonamiento sobre esa estructura — el LLM recorre el índice jerárquico del documento decidiendo, nivel por nivel, qué sección es probable que contenga la respuesta — sin chunking ni indexado vectorial de ningún tipo.

Sin paper académico único que la respalde (Figura 16) — la fuente es el repositorio y la documentación técnica del proyecto, no una publicación peer-reviewed — se cita como tal explícitamente, en vez de forzar una cita que no corresponde.

Fuente: repositorio y documentación técnica de VectifyAI, <https://github.com/VectifyAI/PageIndex>

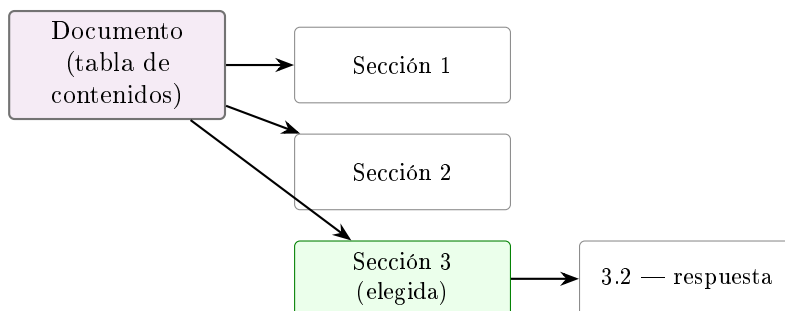


Figura 16: PageIndex: el LLM razona directamente sobre la tabla de contenidos del documento, elige la sección probable y baja un nivel más — sin chunking ni índice vectorial.

5. Era 4 — Arquitecturas multiagente (2023–2025)

Hasta acá, “agente” fue siempre una sola instancia LLM operando sola. Esta era coordina múltiples agentes especializados bajo topologías de orquestación distintas. Aclaración de terminología antes de entrar en detalle: “network”, “Supervisor” y “Jerárquica” son patrones de orquestación popularizados por frameworks (LangGraph, AutoGen), no términos acuñados por un único paper — para cada uno se cita el trabajo académico más cercano, con esa salvedad explícita.

5.1. CAMEL: el primer diálogo nativo entre dos agentes LLM (2023)

Coordinar dos LLMs en una conversación orientada a tarea, sin intervención humana continua, requería guía manual en cada turno. Li et al. [2023] proponen *role-playing* con *inception prompting*: un agente *task specifier* genera la tarea concreta a partir de una idea; dos agentes con roles fijos (asistente, usuario), cada uno con un system message específico de rol y de tarea, conversan autónomamente hasta completarla o alcanzar un criterio de terminación.

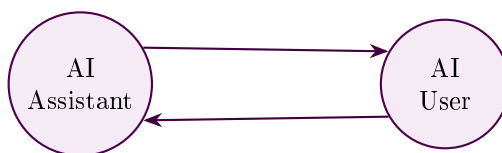


Figura 17: CAMEL: dos agentes con roles fijos, comunicación bidireccional sin coordinador central. Caso mínimo ($N=2$) de topología *network*: cualquier agente puede dirigirse a cualquier otro, sin jerarquía de control.

Salvedad: CAMEL opera con exactamente dos agentes (Figura 17), no con una red de N . Se cita como precursor directo de la topología *network* — agentes pares, sin control central — que frameworks posteriores generalizan a más de dos participantes.

Fuente: [Li et al., 2023]

5.2. AutoGen: conversación multiagente como framework (2023)

Construir un sistema multiagente sin abstracción reutilizable no escala entre proyectos. Wu et al. [2023] proponen *conversable agents* genéricos — customizables vía combinaciones de LLM, input humano y tool execution — orquestables en distintos patrones: chat de a pares, o group chat coordinado por un **GroupChatManager** que decide, turno a turno, qué agente habla a continuación.

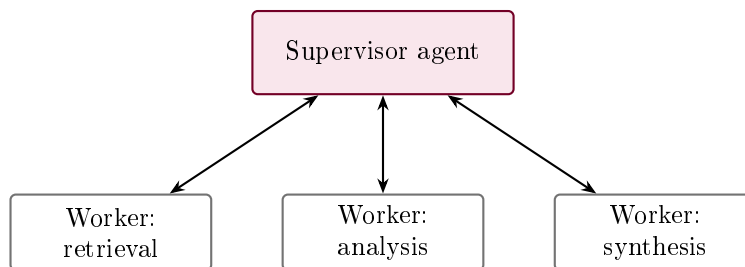


Figura 18: Topología supervisor: orquestador central despacha subtareas a workers especializados e integra sus outputs. Sin comunicación worker-worker directa.

La Figura 18 es el patrón que el material del curso llama “Supervisada”: un orquestador central recibe la tarea completa, despacha subtareas a workers especializados, e integra sus resultados en la respuesta final. AutoGen es el framework académico más cercano a este patrón; el nombre “Supervisor” en sí proviene de la documentación de LangGraph, no de este paper.

Fuente: [Wu et al., 2023]

5.3. MetaGPT: procedimientos operativos estándar para agentes (2023)

Encadenar LLMs sin más estructura que “agente A pasa su output a agente B” acumula error: una alucinación temprana se propaga y se amplifica en cada paso. Hong et al. [2024] codifican Standardized Operating Procedures (SOPs) como secuencias de prompts estructuradas, con roles fijos (product manager, architect, engineer, QA engineer) que producen y validan artefactos intermedios en cada etapa vía un *shared message pool* con mecanismo de subscription — en vez de generar la solución final en un único salto.

MetaGPT es, funcionalmente, un punto intermedio entre esta era y la siguiente: sus roles fijos con responsabilidades acotadas anticipan la especialización que TalkHier formaliza como jerarquía explícita de supervisión.

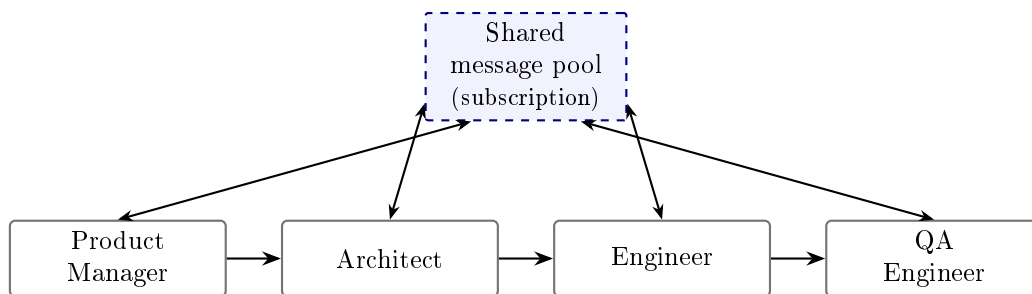


Figura 19: SOP de MetaGPT: roles fijos producen y consumen artefactos intermedios en secuencia, publicando y suscribiéndose a un message pool compartido en vez de pasarse outputs directamente entre sí.

En la Figura 19, cada rol no solo pasa su output al siguiente en la secuencia: publica en el pool compartido, del que cualquier otro rol puede leer según necesite, lo que evita que un cambio de contexto tardío obligue a repetir información ya generada.

Fuente: [Hong et al., 2024]

5.4. AgentVerse: dinámica de grupo emergente (2023)

Chen et al. [2024] modelan colaboración multiagente en cuatro etapas iteradas — expert recruitment, collaborative decision-making, action execution, evaluation — inspiradas en dinámica de grupos humanos, sin imponer una estructura de mando fija a priori. El framework se usa para estudiar qué comportamientos colaborativos emergen de esa dinámica.

Es el precursor cronológico de la jerarquía de supervisión que TalkHier formaliza dos años después, aunque AgentVerse en sí no impone jerarquía anidada estricta — el objeto de estudio es la dinámica de grupo, no la estructura de mando.

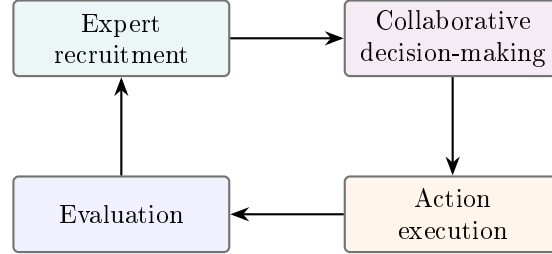


Figura 20: Loop de cuatro etapas de AgentVerse, iterado sin estructura de mando fija a priori: el resultado de la evaluación retroalimenta un nuevo ciclo de reclutamiento de expertos.

El loop de la Figura 20 se itera completo en cada ronda: la composición del grupo de expertos puede cambiar de una ronda a otra según lo que arrojó la evaluación previa, a diferencia de las topologías de control fijo (supervisor, jerárquica) donde los roles quedan determinados de antemano.

Fuente: [Chen et al., 2024]

5.5. TalkHier: supervisores que supervisan supervisores (2025)

Ninguna topología anterior resuelve bien el caso donde el dominio completo es demasiado grande para que un único nivel de coordinación sea viable. Wang et al. [2025] proponen jerarquía anidada explícita — un supervisor de nivel superior delega en supervisores intermedios, cada uno coordinando su propio grupo de workers, con un agente pudiendo ser worker de un equipo y supervisor de otro simultáneamente — combinada con un protocolo de comunicación estructurado (no texto libre) entre niveles.

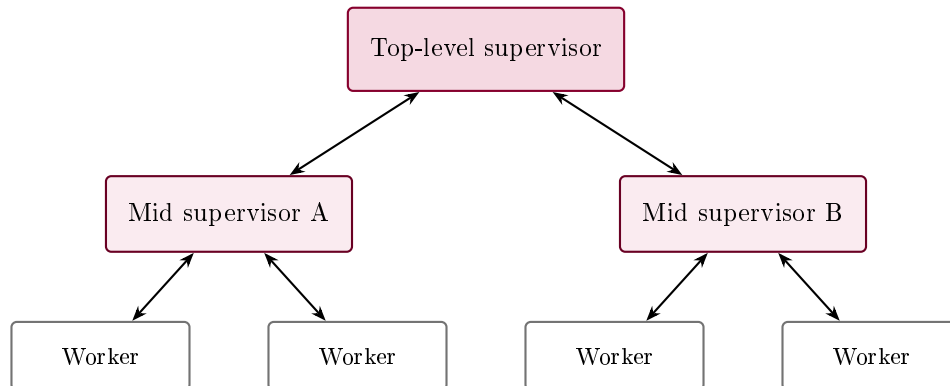


Figura 21: Topología jerárquica: supervisión anidada en múltiples niveles, no solo un nivel plano de especialistas bajo un único supervisor.

Es la formalización más precisa de lo que el material del curso llama “Jerárquica” — más precisa que AgentVerse, su precursor cronológico, que no formaliza esta estructura anidada explícitamente. No confundir con la jerarquía de *resúmenes de contenido* de RAPTOR (Era 3): estructuras análogas — ambas organizan algo en niveles —, pero acá los niveles son de *control entre agentes*, no de abstracción de información.

Fuente: [Wang et al., 2025]

5.6. Mixture-of-Agents: ensamble multiagente por capas (2024)

Las topologías anteriores de esta era organizan agentes por relación de control (quién delega en quién). Wang et al. [2024] proponen una arquitectura ortogonal a eso: capas de agentes donde cada capa — varios LLMs en paralelo, potencialmente de proveedores distintos — produce respuestas independientes a partir de los outputs de la capa anterior, y la capa siguiente actúa como *aggregator*, sintetizando esas respuestas en una versión refinada.

Mixture-of-Agents no compite con las topologías de control anteriores: es una capa de refinamiento por ensamble que se puede insertar sobre cualquiera de ellas — network, supervisor o jerárquica — sin modificar su lógica de coordinación.

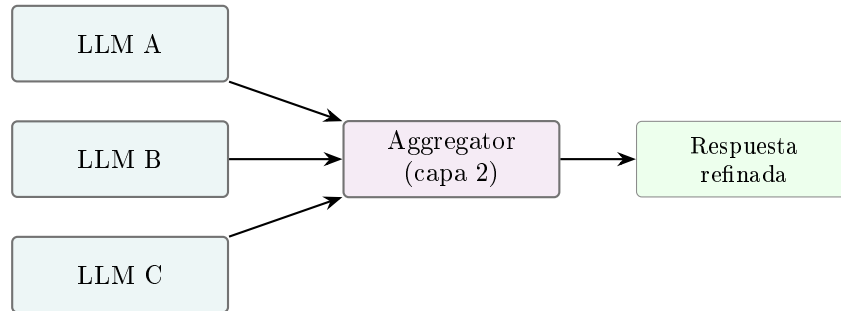


Figura 22: Mixture-of-Agents: varios LLMs en paralelo (capa 1) generan respuestas independientes a la misma entrada; la capa siguiente actúa como aggregator y sintetiza esas respuestas en una versión refinada. El esquema se apila: la salida de un aggregator puede ser la entrada de otra capa de LLMs en paralelo.

La Figura 22 muestra el caso mínimo de dos capas; el paper apila varias capas de proponentes-más-aggregator, cada una refinando la síntesis de la anterior, antes de la respuesta final.

Fuente: [Wang et al., 2024]

6. Era 5 — Protocolos de interoperabilidad (2024–2026)

Las topologías de la Era 4 coordinan agentes dentro de un mismo sistema, con un único diseñador definiendo la topología completa. Esta era resuelve un problema distinto: cómo interoperan agentes contruidos por organizaciones distintas, con frameworks y proveedores de modelo distintos, sin coordinación centralizada de diseño. Ninguna de las dos subsecciones tiene paper académico propio — son especificaciones técnicas de industria, no investigación — y se citan como tales.

6.1. Model Context Protocol (MCP): estandarizar agente↔herramientas (2024)

Sin un estándar común, cada aplicación LLM reimplementa su propia integración ad-hoc con cada fuente de datos o herramienta externa — un problema de integración $N \times M$ (N aplicaciones,

M fuentes). Anthropic publica MCP: un protocolo abierto cliente-servidor sobre JSON-RPC, con arquitectura host-client-server, que estandariza cómo una aplicación LLM (el host) descubre y consume *resources*, *tools* y *prompts* expuestos por servidores MCP — reduciendo la integración a $N+M$. Adoptado rápidamente más allá de su origen (OpenAI, Google DeepMind, entre otros), es hoy el estándar de facto para tool use estructurado.

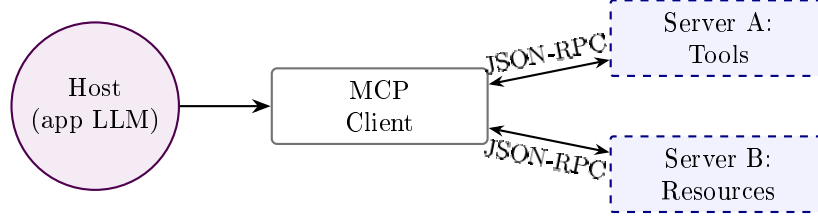
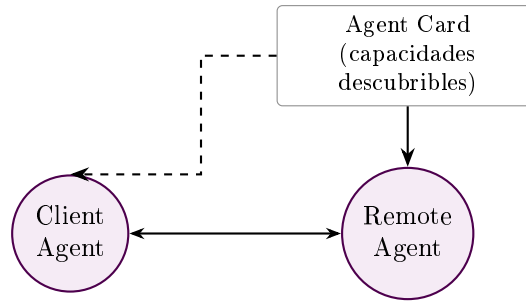


Figura 23: MCP: el host embebe un client por cada server con el que necesita hablar; el client habla JSON-RPC con cada server, que expone sus tools, resources y prompts con una interfaz uniforme — de ahí la reducción de $N \times M$ integraciones a $N + M$.

Fuente: Anthropic, especificación técnica, <https://www.anthropic.com/news/model-context-protocol> (no es un paper de investigación)

6.2. Agent2Agent (A2A): estandarizar agente↔agente (2025–2026)

MCP resuelve agente↔herramienta; queda sin resolver agente↔agente entre organizaciones distintas. Google publica A2A: agentes se anuncian vía *Agent Cards* — manifiestos de capacidades descubribles —, negocian y ejecutan tareas con un ciclo de vida de estados explícito (submitted, working, input-required, completed), sobre HTTP, Server-Sent Events y JSON-RPC. Anunciado en abril de 2025, alcanza v1.0 en abril de 2026 con gobernanza cedida a la Linux Foundation y más de 150 organizaciones adheridas.



submitted → working → input-required → completed

Figura 24: A2A: el client agent descubre las capacidades del remote agent vía su Agent Card (línea punteada), y luego intercambia tareas con él sobre HTTP/SSE/JSON-RPC; cada tarea recorre un ciclo de vida de estados explícito hasta completarse.

Fuente: Google / Linux Foundation, especificación técnica (no es un paper de investigación). Análisis académico de terceros sobre el protocolo: [Louck et al., 2025, Kang and Diponegoro, 2026]

7. Era 6 — El campo se sistematiza (2025–2026)

Ninguna arquitectura nueva en esta era: en su lugar, los primeros intentos de sistematizar todo lo anterior en una taxonomía formal — señal de que el campo pasó de una sucesión de propuestas puntuales a un cuerpo de conocimiento comparable y evaluable. Ambas subsecciones son surveys, no papers de arquitectura: se citan como marco conceptual, no como origen de ninguna técnica específica.

7.1. Agentic RAG: primera taxonomía formal (2025)

Singh et al. [2025] sistematizan las variantes de RAG agéntico de la Era 3 (reflection, planning, tool use, multi-agent collaboration) bajo una taxonomía común de patrones de diseño aplicados a RAG — el primer marco de referencia para hablar de “Agentic RAG” y “Autonomous RAG” como categorías, y no solo como técnicas individuales sin relación explícita entre sí.

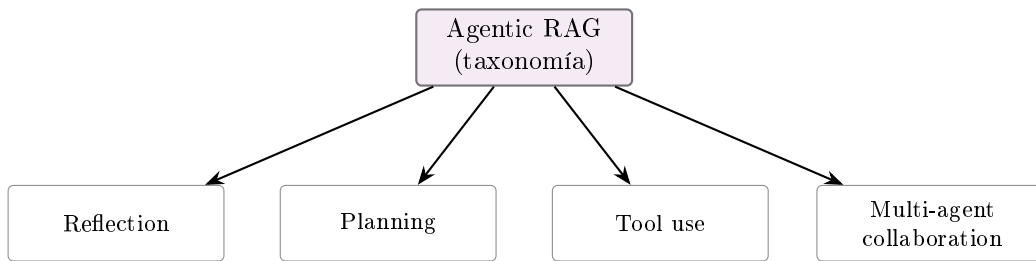


Figura 25: Taxonomía de Singh et al. [2025]: los patrones de diseño de la Era 3 (Self-RAG/Reflection, RAPTOR/Adaptive-RAG, MemGPT, arquitecturas multiagente de la Era 4) quedan agrupados como categorías de una misma familia, no como técnicas inconexas.

Fuente: [Singh et al., 2025]

7.2. SoK: Agentic RAG como proceso de decisión (2026)

Mishra et al. [2026] van un paso más allá: formalizan el loop de razonamiento-recuperación agéntico como un proceso de decisión de Markov parcialmente observable (POMDP) de horizonte finito, con una taxonomía sobre estrategias de planificación, orquestación de retrieval, paradigmas de memoria y coordinación de herramientas. Es la pieza más reciente confirmada del timeline (marzo de 2026) y cierra la parte del artículo anclada en papers verificados, antes de la sección prospectiva.

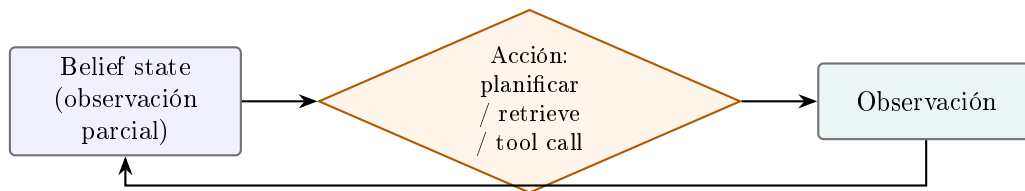


Figura 26: Formalización POMDP de Mishra et al. [2026]: en cada paso el agente actúa (planifica, recupera o invoca una herramienta) bajo observabilidad parcial del estado real, recibe una observación, y actualiza su belief state — horizonte finito, iterado hasta una condición de parada.

Fuente: [Mishra et al., 2026]

8. Hacia dónde va esto

Nota editorial

Esta sección es lectura/opinión informada de los autores, a diferencia del resto del artículo, que ancla cada arquitectura en un paper verificado con fecha de publicación real. Se señala así explícitamente para que quede clara la distinción entre lo evidenciado y lo proyectado. Las señales que siguen son papers de 2025–2026 encontrados durante el research de este proyecto, no predicciones sin respaldo — pero por definición todavía no son “arquitecturas” consolidadas con la misma certeza que las de las secciones anteriores.

- **Gobernanza y accountability inter-organizacional sin resolver** — ni MCP ni A2A especifican membership, votación, preservación de disenso o escalamiento humano; ningún protocolo de interoperabilidad actual cubre quién responde legalmente cuando un agente de una organización actúa sobre datos de otra [Kang and Diponegoro, 2026].
- **Seguridad multiagente como subcampo propio** — superficie de ataque cualitativamente distinta a la de un sistema single-agent: memory poisoning, trust exploitation entre agentes, y vulnerabilidades específicas de la composición agent-tool [Nguyen et al., 2026].
- **Infraestructura para una “Agentic Web”** — discovery semántico de agentes, memoria persistente a nivel de plataforma, primitivas económicas para transacciones agente-a-agente sin humano en el loop [Dey and Viradacha, 2026].
- **Neural architecture search aplicado a topologías de agentes** — en vez de fijar network/supervisor/jerárquica a mano, el sistema optimiza su propia topología de orquestación como distribución continua sobre un *agentic supernet*, condicionada a la query [Zhang et al., 2025].

Las cuatro señales apuntan en la misma dirección: el eje de diseño se desplaza de *qué arquitectura elegir* a *qué tan poco control de diseño hace falta fijar a mano* — topología, memoria, gobernanza y hasta la propia búsqueda arquitectónica, delegadas cada vez más al sistema.

9. Conclusión

Cada era de este timeline le saca control de diseño a un pipeline fijo y se lo devuelve al propio sistema. RAG (Era 1) resuelve el corte de conocimiento con un pipeline de dos fases sin ningún grado de libertad. La Era 2 le da al modelo control sobre su propio control flow — decide qué acción ejecutar en cada iteración. La Era 3 aplica ese control específicamente al pipeline de RAG: cuándo recuperar, si lo recuperado es confiable, con qué profundidad iterar. La Era 4 distribuye esa agencia entre múltiples agentes especializados, bajo topologías de orquestación que van de red plana a jerarquía anidada. La Era 5 estandariza cómo esa agencia distribuida interopera entre organizaciones distintas. La Era 6 formaliza todo lo anterior como un objeto de estudio sistemático — una taxonomía, un proceso de decisión POMDP — en vez de una colección de técnicas puntuales.

La tendencia que atraviesa las seis eras no es que una arquitectura particular “gane” sobre las demás: es una escalada continua en cuánta decisión de diseño se delega del pipeline al modelo, y del modelo individual a la red de agentes. Las señales prospectivas de la Sección 8 — gobernanza, seguridad, infraestructura de descubrimiento, arquitecturas que se optimizan a sí mismas — son, leídas en conjunto, la continuación directa de esa misma escalada.

Referencias

- Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations (ICLR)*, 2024.
- Brian J Chan, Chao-Ting Chen, Jui-Hung Cheng, and Hen-Hsen Huang. Don’t do RAG: When cache-augmented generation is all you need for knowledge tasks. In *Companion Proceedings of the ACM Web Conference (WWW)*, 2025.
- Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu, Yi-Hsin Hung, Chen Qian, Yujia Qin, Xin Cong, Ruobing Xie, Zhiyuan Liu, Maosong Sun, and Jie Zhou. AgentVerse: Facilitating multi-agent collaboration and exploring emergent behaviors. In *International Conference on Learning Representations (ICLR)*, 2024.
- Robin Dey and Panyanon Viradecha. Infrastructure for the agentic web: Gap analysis and architecture from the agentverse platform. *arXiv preprint arXiv:2606.20570*, 2026.
- Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.
- Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. Precise zero-shot dense retrieval without relevance labels. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023a.
- Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Qianyu Guo, Meng Wang, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2023b.
- Yunfan Gao, Yun Xiong, Meng Wang, and Haofen Wang. Modular RAG: Transforming RAG systems into LEGO-like reconfigurable frameworks. *arXiv preprint arXiv:2407.21059*, 2024.
- Sirui Hong, Xiawu Zheng, Jonathan Chen, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*, 2024.
- Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C. Park. Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2024.
- Richard Kang and Yudho Diponegoro. Governance gaps in agent interoperability protocols: What MCP, A2A, and ACP cannot express. *arXiv preprint arXiv:2606.31498*, 2026.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474, 2020.

- Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. CAMEL: Communicative agents for “mind” exploration of large language model society. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Yedidel Louck, Ariel Stulman, and Amit Dvir. Improving google A2A protocol: Protecting sensitive data and mitigating unintended harms in multi-agent systems. *arXiv preprint arXiv:2505.12490*, 2025.
- Saroj Mishra, Suman Niroula, Umesh Yadav, Dilip Thakur, Srijan Gyawali, and Shiva Gaire. SoK: Agentic retrieval-augmented generation (RAG): Taxonomy, architectures, evaluation, and research directions. *arXiv preprint arXiv:2603.07379*, 2026.
- Tam Nguyen, Moses Ndebugre, and Dheeraj Arremsetty. Security considerations for multi-agent systems. *arXiv preprint arXiv:2603.09002*, 2026.
- Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023.
- Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D. Manning. RAPTOR: Recursive abstractive processing for tree-organized retrieval. In *International Conference on Learning Representations (ICLR)*, 2024.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Aditi Singh, Abul Ehtesham, Saket Kumar, and Tala Talaie Khoei. Agentic retrieval-augmented generation: A survey on agentic RAG. *arXiv preprint arXiv:2501.09136*, 2025.
- Junlin Wang, Jue Wang, Ben Athiwaratkun, Ce Zhang, and James Zou. Mixture-of-agents enhances large language model capabilities. *arXiv preprint arXiv:2406.04692*, 2024.
- Zhao Wang, Sota Moriyama, Wei-Yao Wang, Briti Gangopadhyay, and Shingo Takamatsu. Talk structurally, act hierarchically: A collaborative framework for LLM multi-agent systems. In *arXiv preprint*, 2025.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.
- Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. Corrective retrieval augmented generation. *arXiv preprint arXiv:2401.15884*, 2024.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- Guibin Zhang, Luyang Niu, Junfeng Fang, Kun Wang, Lei Bai, and Xiang Wang. Multi-agent architecture search via agentic supernet. *arXiv preprint arXiv:2502.04180*, 2025.